

Министерство науки и высшего образования
Пензенский государственный университет
Кафедра "Вычислительная техника"

ОТЧЕТ

по лабораторной работе №9

по курсу "Логика и основы алгоритмизации в инженерных задачах"

на тему "Поиск расстояний в графе"

Выполнил:

студенты группы 24ВВВ1

Куничкина В.А.

Суркова Д.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Задание 1

- 1 Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
- 2 Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.
- 3.* Реализуйте процедуру поиска расстояний для графа, представленного списками смежности.

Задание 2*

- 1 Реализуйте процедуру поиска расстояний на основе обхода в глубину.
- 2 Реализуйте процедуру поиска расстояний на основе обхода в глубину для графа, представленного списками смежности.
- 3 Оцените время работы реализаций алгоритмов поиска расстояний на основе обхода в глубину и обхода в ширину для графов разных порядков.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <limits.h>
#include <time.h>

#define INF INT_MAX

typedef struct {
    int* items;
    int front;
    int rear;
```

```

        int capacity;
    } Queue;

void initQueue(Queue* q, int capacity) {
    q->items = (int*)malloc(capacity * sizeof(int));
    q->front = -1;
    q->rear = -1;
    q->capacity = capacity;
}

void freeQueue(Queue* q) {
    free(q->items);
}

int isEmpty(Queue* q) {
    return q->front == -1;
}

void enqueue(Queue* q, int value) {
    if (q->rear == q->capacity - 1) {
        printf("Очередь переполнена\n");
    }
    else {
        if (q->front == -1) {
            q->front = 0;
        }
        q->rear++;
        q->items[q->rear] = value;
    }
}

int dequeue(Queue* q) {
    int item;
    if (isEmpty(q)) {
        printf("Очередь пуста\n");
        item = -1;
    }
}

```

```

    }
    else {
        item = q->items[q->front];
        q->front++;
        if (q->front > q->rear) {
            q->front = q->rear = -1;
        }
    }
    return item;
}

int** createGraph(int n) {
    int** graph = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        graph[i] = (int*)calloc(n, sizeof(int));
    }
    return graph;
}

int** createDistanceMatrix(int n) {
    int** distMatrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        distMatrix[i] = (int*)malloc(n * sizeof(int));
        for (int j = 0; j < n; j++) {
            if (i == j) {
                distMatrix[i][j] = 0;
            }
            else {
                distMatrix[i][j] = INF;
            }
        }
    }
    return distMatrix;
}

void freeGraph(int** graph, int n) {

```

```

    for (int i = 0; i < n; i++) {
        free(graph[i]);
    }
    free(graph);
}

void freeDistanceMatrix(int** distMatrix, int n) {
    for (int i = 0; i < n; i++) {
        free(distMatrix[i]);
    }
    free(distMatrix);
}

void generateUndirectedGraph(int** graph, int n, int isWeighted) {

    int minEdges = n - 1;
    int maxEdges = n * (n - 1) / 2;
    int targetEdges = minEdges + rand() % (maxEdges - minEdges + 1);

    int edgesAdded = 0;

    for (int i = 1; i < n; i++) {
        int j = rand() % i;
        if (isWeighted) {
            graph[i][j] = rand() % 10 + 1;
            graph[j][i] = graph[i][j];
        }
        else {
            graph[i][j] = 1;
            graph[j][i] = 1;
        }
        edgesAdded++;
    }

    while (edgesAdded < targetEdges) {
        int i = rand() % n;

```

```

    int j = rand() % n;

    if (i != j && graph[i][j] == 0) {
        if (isWeighted) {
            graph[i][j] = rand() % 10 + 1;
            graph[j][i] = graph[i][j];
        }
        else {
            graph[i][j] = 1;
            graph[j][i] = 1;
        }
        edgesAdded++;
    }
}

void generateDirectedGraph(int** graph, int n, int isWeighted) {
    int minEdges = n - 1;
    int maxEdges = n * (n - 1);
    int targetEdges = minEdges + rand() % (maxEdges - minEdges + 1);

    int edgesAdded = 0;

    for (int i = 1; i < n; i++) {
        int j = rand() % i;
        if (rand() % 2 == 0) {
            if (isWeighted) {
                graph[i][j] = rand() % 10 + 1;
            }
            else {
                graph[i][j] = 1;
            }
        }
        else {
            if (isWeighted) {
                graph[j][i] = rand() % 10 + 1;
            }

```

```

        }
        else {
            graph[j][i] = 1;
        }
    }
    edgesAdded++;
}

while (edgesAdded < targetEdges) {
    int i = rand() % n;
    int j = rand() % n;

    if (i != j && graph[i][j] == 0) {
        if (isWeighted) {
            graph[i][j] = rand() % 10 + 1;
        }
        else {
            graph[i][j] = 1;
        }
        edgesAdded++;
    }
}
}

```

```

void printMatrix(int** graph, int n) {
    printf("Матрица смежности:\n");
    printf("  ");
    for (int i = 0; i < n; i++) {
        printf("%3d", i);
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%3d", i);
        for (int j = 0; j < n; j++) {

```

```

        printf("%3d", graph[i][j]);
    }
    printf("\n");
}
printf("\n");
}

```

```

void printDistanceMatrix(int** distMatrix, int n) {
    printf("Матрица дальности (расстояний):\n");
    printf("  ");
    for (int i = 0; i < n; i++) {
        printf("%5d", i);
    }
    printf("\n");

```

```

    for (int i = 0; i < n; i++) {
        printf("%3d", i);
        for (int j = 0; j < n; j++) {
            if (distMatrix[i][j] == INF) {
                printf("  INF");
            }
            else {
                printf("%5d", distMatrix[i][j]);
            }
        }
        printf("\n");
    }
    printf("\n");
}

```

```

void findDistancesUnweighted(int** graph, int n, int start, int* distances) {
    Queue q;
    initQueue(&q, n);

    for (int i = 0; i < n; i++) {
        distances[i] = -1;
    }

```



```

    }

    distances[start] = 0;
    enqueue(&q, start);

    while (!isEmpty(&q)) {
        int current = dequeue(&q);

        for (int i = 0; i < n; i++) {
            if (graph[current][i] != 0 && distances[i] == -1) {
                distances[i] = distances[current] + 1;
                enqueue(&q, i);
            }
        }
    }

    freeQueue(&q);
}

void findDistancesWeighted(int** graph, int n, int start, int* distances) {
    int* visited = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        distances[i] = INF;
    }
    distances[start] = 0;

    for (int count = 0; count < n - 1; count++) {
        int min = INF, min_index = -1;
        for (int v = 0; v < n; v++) {
            if (!visited[v] && distances[v] <= min) {
                min = distances[v];
                min_index = v;
            }
        }
    }
}

```

```

        }
    }

    if (min_index == -1) break;

    visited[min_index] = 1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && graph[min_index][v] != 0 &&
            distances[min_index] != INF &&
            distances[min_index] + graph[min_index][v] < distances[v]) {
            distances[v] = distances[min_index] + graph[min_index][v];
        }
    }
}

free(visited);
}

void buildDistanceMatrixFloydWarshall(int** graph, int n, int** distMatrix, int
isWeighted) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                distMatrix[i][j] = 0;
            }
            else if (graph[i][j] != 0) {
                distMatrix[i][j] = isWeighted ? graph[i][j] : 1;
            }
            else {
                distMatrix[i][j] = INF;
            }
        }
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {

```

```

        for (int j = 0; j < n; j++) {
            if (distMatrix[i][k] != INF && distMatrix[k][j] != INF &&
                distMatrix[i][k] + distMatrix[k][j] < distMatrix[i][j]) {
                distMatrix[i][j] = distMatrix[i][k] + distMatrix[k][j];
            }
        }
    }
}

```

```

void findEccentricitiesFromDistanceMatrix(int** distMatrix, int n, int* eccentricities)
{
    for (int i = 0; i < n; i++) {
        eccentricities[i] = -1;
        for (int j = 0; j < n; j++) {
            if (i != j && distMatrix[i][j] != INF) {
                if (distMatrix[i][j] > eccentricities[i]) {
                    eccentricities[i] = distMatrix[i][j];
                }
            }
        }
    }
}

```

// ГАРАНТИРОВАННО 2 ЦЕНТРА ТЯЖЕСТИ

```

void findGraphMedian(int** distMatrix, int n) {
    int* sumDistances = (int*)calloc(n, sizeof(int));
    int minSum = INF;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (distMatrix[i][j] != INF) {
                sumDistances[i] += distMatrix[i][j];
            }
        }
        if (sumDistances[i] < minSum) {
            minSum = sumDistances[i];
        }
    }
}

```

```
    }  
}
```

```
printf("Центр тяжести: ");  
int count = 0;  
for (int i = 0; i < n; i++) {  
    if (sumDistances[i] == minSum) {  
        if (count > 0) printf(", ");  
        printf("%d", i);  
        count++;  
    }  
}
```

```
free(sumDistances);  
}  
  
void analyzeGraphFromDistanceMatrix(int** distMatrix, int n, int isDirected) {  
    int* eccentricities = (int*)malloc(n * sizeof(int));  
    findEccentricitiesFromDistanceMatrix(distMatrix, n, eccentricities);  
  
    printf("Эксцентриситеты вершин:\n");  
    for (int i = 0; i < n; i++) {  
        if (eccentricities[i] == -1) {  
            printf("Вершина %d: недостижимы другие вершины\n", i);  
        }  
        else {  
            printf("Вершина %d: %d\n", i, eccentricities[i]);  
        }  
    }  
    printf("\n");  
  
    int radius = INF;  
    int diameter = -1;  
  
    for (int i = 0; i < n; i++) {
```

```

    if (eccentricities[i] != -1) {
        if (eccentricities[i] < radius) {
            radius = eccentricities[i];
        }
        if (eccentricities[i] > diameter) {
            diameter = eccentricities[i];
        }
    }
}

if (radius == INF) {
    printf("Граф не связный – невозможно определить радиус и диаметр\n");
}
else {
    printf("Радиус графа: %d\n", radius);
    printf("Диаметр графа: %d\n", diameter);

    printf("Центральные вершины: ");
    for (int i = 0; i < n; i++) {
        if (eccentricities[i] == radius) {
            printf("%d ", i);
        }
    }
    printf("\n");

    printf("Периферийные вершины: ");
    for (int i = 0; i < n; i++) {
        if (eccentricities[i] == diameter) {
            printf("%d ", i);
        }
    }
    printf("\n");
}

findGraphMedian(distMatrix, n);
printf("\n");

```

```

    free(eccentricities);
}

void printDistances(int* distances, int n, int start) {
    printf("Расстояния от вершины %d:\n", start);
    for (int i = 0; i < n; i++) {
        if (distances[i] == INF || distances[i] == -1) {
            printf("До вершины %d: недостижима\n", i);
        }
        else {
            printf("До вершины %d: %d\n", i, distances[i]);
        }
    }
    printf("\n");
}

int safeInputInt(const char* prompt, int maxValue) {
    char buffer[100];
    int success = 0;
    int n = 0;

    while (!success) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            printf("Ошибка ввода!\n");
            continue;
        }

        int onlyDigits = 1;
        int len = strlen(buffer);

        if (len > 0 && buffer[len - 1] == '\n') {
            buffer[len - 1] = '\0';
            len--;

```

```

    }

    for (int i = 0; i < len; i++) {
        if (!isdigit(buffer[i])) {
            onlyDigits = 0;
            break;
        }
    }

    if (!onlyDigits || len == 0) {
        printf("Ошибка: введите только цифры!\n");
        continue;
    }

    char* endptr;
    n = strtol(buffer, &endptr, 10);

    if (n < 1 || n > maxValue) {
        printf("Ошибка: введите число от 1 до %d!\n", maxValue);
        continue;
    }

    success = 1;
}

return n;
}

int inputStartVertex(int n) {
    char buffer[100];
    int success = 0;
    int vertex = 0;

    while (!success) {
        printf("Введите стартовую вершину (0 до %d): ", n - 1);
    }
}

```

```

if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
    printf("Ошибка ввода!\n");
    continue;
}

int onlyDigits = 1;
int len = strlen(buffer);

if (len > 0 && buffer[len - 1] == '\n') {
    buffer[len - 1] = '\0';
    len--;
}

for (int i = 0; i < len; i++) {
    if (!isdigit(buffer[i])) {
        onlyDigits = 0;
        break;
    }
}

if (!onlyDigits || len == 0) {
    printf("Ошибка: введите только цифры!\n");
    continue;
}

char* endptr;
vertex = strtol(buffer, &endptr, 10);

if (vertex < 0 || vertex >= n) {
    printf("Ошибка: вершина должна быть от 0 до %d!\n", n - 1);
    continue;
}

success = 1;
}

```



```

        return vertex;
    }

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");
    srand((unsigned int)time(NULL));

    int n = 0;
    int isWeighted = 1;
    int startVertex = 0;

    n = safeInputInt("Количество вершин (1-10): ", 10);

    startVertex = inputStartVertex(n);

    printf("\n=== АНАЛИЗ НЕОРИЕНТИРОВАННОГО ГРАФА ===\n");
    printf("Количество вершин: %d\n", n);
    printf("Тип графа: Неориентированный взвешенный\n");

    int** graphUndirected = createGraph(n);
    int** distMatrixUndirected = createDistanceMatrix(n);
    int* distancesUndirected = (int*)malloc(n * sizeof(int));

    generateUndirectedGraph(graphUndirected, n, isWeighted);
    printMatrix(graphUndirected, n);

    printf("Построение матрицы дальности...\n");
    buildDistanceMatrixFloydWarshall(graphUndirected, n, distMatrixUndirected,
isWeighted);
    printDistanceMatrix(distMatrixUndirected, n);

    printf("Стартовая вершина для анализа: %d\n\n", startVertex);

    if (isWeighted) {
        findDistancesWeighted(graphUndirected, n, startVertex, distancesUndirected);
    }
    else {

```

```

        findDistancesUnweighted(graphUndirected, n, startVertex, distancesUndirected);
    }

    printDistances(distancesUndirected, n, startVertex);

    printf("=== АНАЛИЗ НЕОРИЕНТИРОВАННОГО ГРАФА ИЗ МАТРИЦЫ ДАЛЬНОСТИ ===\n");
    analyzeGraphFromDistanceMatrix(distMatrixUndirected, n, 0);

    printf("\n=== АНАЛИЗ ОРИЕНТИРОВАННОГО ГРАФА ===\n");
    printf("Количество вершин: %d\n", n);
    printf("Тип графа: Ориентированный взвешенный\n");

    int** graphDirected = createGraph(n);
    int** distMatrixDirected = createDistanceMatrix(n);
    int* distancesDirected = (int*)malloc(n * sizeof(int));

    generateDirectedGraph(graphDirected, n, isWeighted);
    printMatrix(graphDirected, n);

    printf("Построение матрицы дальности...\n");
    buildDistanceMatrixFloydWarshall(graphDirected, n, distMatrixDirected, isWeighted);
    printDistanceMatrix(distMatrixDirected, n);

    printf("Стартовая вершина для анализа: %d\n\n", startVertex);

    if (isWeighted) {
        findDistancesWeighted(graphDirected, n, startVertex, distancesDirected);
    }
    else {
        findDistancesUnweighted(graphDirected, n, startVertex, distancesDirected);
    }

    printDistances(distancesDirected, n, startVertex);

    printf("АНАЛИЗ ОРИЕНТИРОВАННОГО ГРАФА ИЗ МАТРИЦЫ ДАЛЬНОСТИ\n");

```

```
analyzeGraphFromDistanceMatrix(distMatrixDirected, n, 1);

free(distancesUndirected);
free(distancesDirected);
freeGraph(graphUndirected, n);
freeGraph(graphDirected, n);
freeDistanceMatrix(distMatrixUndirected, n);
freeDistanceMatrix(distMatrixDirected, n);

getchar();
return 0;
}
```

Результат работы программы

```
Введите стартовую вершину (0-4): 1

1.2: Поиск расстояний в ширину (матрица смежности)
Порядок обхода вершин (матрица): 1 0 2 3 4

Расстояния от вершины 1:
Вершина 0: расстояние 1
Вершина 1: расстояние 0
Вершина 2: расстояние 2
Вершина 3: расстояние 2
Вершина 4: расстояние 3
Время выполнения: 3,99 мс
Количество операций: 34

1.3: Поиск расстояний (списки смежности)
Порядок обхода вершин (списки): 1 0 3 2 4

Расстояния от вершины 1:
Вершина 0: расстояние 1
Вершина 1: расстояние 0
Вершина 2: расстояние 2
Вершина 3: расстояние 2
Вершина 4: расстояние 3
Время выполнения: 1,91 мс
Количество операций: 21

2.1: Поиск расстояний в глубину (DFS)
Порядок обхода вершин (DFS): 1 0 2 3 4
Расстояния от вершины 1:
Вершина 0: расстояние 1
Вершина 1: расстояние 0
Вершина 2: расстояние 2
Вершина 3: расстояние 3
Вершина 4: расстояние 4
Время выполнения: 3,28 мс
Количество операций: 34

2.2: Поиск расстояний (DFS списки)
Порядок обхода вершин (DFS списки): 1 0 3 4 2
Расстояния от вершины 1:
Вершина 0: расстояние 1
Вершина 1: расстояние 0
Вершина 2: расстояние 4
Вершина 3: расстояние 2
Вершина 4: расстояние 3
Время выполнения: 3,34 мс
Количество операций: 21
```

Вывод: В ходе лабораторной работы были получены навыки работы с поиском расстояний в графах.